# Sliding Window

**Minimum Size Subarray Sum → Medium**

**Longest Substring Without Repeating Characters → Medium**

**Substring with Concatenation of All Words → Hard(pending)**

**Minimum Window Substring → Hard (pending)**

## 209. Minimum Size Subarray Sum

Given an array of positive integers `nums` and a positive integer `target`, return *the **minimal length** of a subarray whose sum is greater than or equal to* `target`. If there is no such subarray, return `0` instead.

**Example 1:**

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem con
straint.
```

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int n = nums.size();

        int i = 0, j = 0;

        int sum = 0;
        int minL = n+1;

        while(j < n) {
            sum += nums[j];
```

```
        while(sum >= target) {
            minL = min(minL, j-i+1);
            sum -= nums[i];
            i++;
        }

        j++;
    }
    return minL == n+1 ? 0 : minL;
  }
};
```

# 209. Minimum Size Subarray Sum

i/p: target = 7, nums = {2, 3, 1, 2, 4, 3}

o/p: 2

minimum length subarray whose sum = 7 (4+3) ↳ target

Hint for sliding window questions
1. continuous
2. contiguous subarray

eg {(2, 2, 3), 1, 2, (4, 3)}

sum = 8; sum = 7; sum = 7

Take min length of subarray
o/p = 2 → length

⇒ Dry Run — Two Pointer approach

target = 7, | 2 | 3 | 1 | 2 | 4 | 3 | (0 1 2 3 4 5) → length (n) = 6

i, j

minlength = 6 → n
sum = 0

Now,
sum = 0 + 2
sum = 2

Now check this sum is equal to or greater than target (7) or not
2 ≥ 7 ✗ (sum ≥ target)
No, So, j++

| 2 | 3 | 1 | 2 | 4 | 3 |
i j

sum = 2 + 3 = 5 (previous sum)
5 ≥ 7 (No)
So, j++

target = 7, | 2 | 3 | 1 | 2 | 4 | 3 |
i j

sum = 5 + 1 = 6 (j) 6 ≥ 7 (No) So, j++

| 2 | 3 | 1 | 2 | 4 | 3 |
i j

window

| 2 | 3 | 1 | 2 | 4 | 3 |
i j

sum = 6 + 2 = 8 8 ≥ 7 (Yes)
length = 4

| 2 | 3 | 1 | 2 | 4 | 3 |
i j

Try to minimize length so move i → i++

sum = 8 − 2 (i) value = 6 (sum = 6)

6 ≥ 7 (target) → NO
↳ so we can't reduce the length
So go to j → j++

window

| 2 | 3 | 1 | 2 | 4 | 3 |
i j

j++
sum = 6 + 4 (j) = 10

10 ≥ 7 (Yes)
So try to reduce the length of array
i.e क्या हम इस length को कम कर सकते हैं, और reduce कर के क्या sum target के equal पा सकते हैं।

So move i → i++

target = 7

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | i | | | j | |

sum = 10 - 3 = 7

7 ≥ target (yes)

$\boxed{i++}$

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | i | | | j | |

~~sum/2~~

So, by reducing length still get target sum

minLength = ~~6~~ 3

So, $\boxed{\text{minLength} = 3}$

$(j-i+1) \Rightarrow = 4-2+1$

$= 2+1$

$= \boxed{3}$

Now, $\boxed{i++}$

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | j | | |

sum = 7 - 1 ↳ i

sum = 6

6 ≥ target (No)

So move j

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | | j | |

sum = 6 + 3 = 9

9 ≥ target ✓

minLength = j - i + 1

$\boxed{\text{minL} = 3}$

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | | j | |

try to reduce length

i++

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | | j | |

sum = 9 - 2 = 7

7 ≥ target ✓

$\boxed{\text{minL} = 2}$

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | | j | |

sum = 7 - 4

$\boxed{\text{sum} = 3}$

3 ≥ target (No) j++

| 2 | 3 | 1 | 2 | 4 | 3 |
|---|---|---|---|---|---|
| | | i | | | j |

out of bound so stop

T.C = O(n) , S.C = O(1)

code

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int> & nums) {
        int n = nums.size();
        int i = 0, j = 0;
        ~~int minL = n+1~~ int minL = INT_MAX;   (or) n+1
        int sum = 0;
        while(j < n){
            sum += nums[j];
            → try to reduce the length of window.
            while(sum >= target) {
                minL = min(minL , j-i+1);
                sum -= nums[i];
                i++; → right shift of i
            }
            j++;
        }
        ~~return minL;~~ ⇒ return minL == INT_MAX ? 0 : minL;   (or) n+1
    };
}
```

## 3. Longest Substring Without Repeating Characters

Given a string `s`, find the length of the **longest substring** without duplicate characters.

**Example 1:**

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "ca
b" are also correct answers.
```

```
// TC: O(n)    ,
//SC: O(n) (worst case)
//For lowercase/ASCII characters → O(1)  → S.C
//In worst case (all unique characters) → O(n) → S.C
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> charSet;

        int maxLength = 0;
        int left = 0;

        for (int right = 0; right < s.length(); right++) {

            while (charSet.count(s[right])) {
                charSet.erase(s[left]);
                left++;
            }

            charSet.insert(s[right]);
            maxLength = max(maxLength, right - left + 1);
        }

        return maxLength;
```

```
    }
};
                    //  OR

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int largest=0;
        for(int i=0; i<s.size() ;i++){
            unordered_map<char,bool> st;
            int len=0;
            for(int j=i;j<s.size();j++){
                if(st[s[j]] ==true){
                    break;
                }
                 len++;
                 st[s[j]]=true;
            }
            largest=max(len,largest);
        }
        return largest;
    }
};
```

## 3. Longest Repeating Substring without repeating Characters

↳ continuous

eg: S = "abcabcbb"

abc
cab → length = 3 Ans
bca —

eg S = "bbbbb"

b
1 → length
Ans

eg S = "pwwkew"

wke
length = 3 Ans

**Brute Force**

S = "pwwkewxpw"

start iterating until get repeating character

pwwkewxpx
↑ ↑ ↑
i i i ↳ repeating so stop

So, pw → length = 2

S = "pwwkewxpw"

pwwkewxpw
↑ ↑
i i ↳ repeating so stop

w → length = 1
↳ short length so ignore

S = "pwwkewxpw"
↑ ↑ ↑ ↑
i i i i

wke → length = 3

S = "pwwkewxpw"

kewxp → length = 5

longest substring o/p

## optimal Solution → 2pointer approach

S = "pwwkewxpw"

left, right

check p is present in set or not if Not present so put in set

Set → [p]

then right ++

S = pwwkewxpw

left, right

Set = [p w]

right ++

S = pwwkewxpw

left, right

w is present in set so, we have duplicate

So, window with no duplicate = right - left

S = pwwkewxpw

length = 2

maxlength = 2

⇒ S = pwwkewxpx

left, right

remove p from set remove left and then left ++

⇒ S = pwwkewxpx

left, right

Set ⇒ [w]

S = pwwkewxpw

left, right

our aim to remove duplicate so,

[left ++]

S = pwwkewxpw

left, right

Set = [ ]

S = "pwwkewxpw"

left, right

w is not present in set so, put w in set

Set = [w]

right ++

S = "pwwkewxpw"

left, right

Set = [w k]

right ++

S = pwwkewxpw

left, right

Set = [w, k, e]

right ++

0 1 2 3 4 5 6 7 8

S = pwwkewxpw

left, right

w is already present in set

window with no duplicate = right - left

window = 5 - 2 = 3

[length = 3] → [maxLen = 3]

left ++

S = pwwkewxpw

left, right

Set = [k, e]

S = pwwkewxpw

left, right

w is not present in set so insert it

Set = [k, e, w]

right ++

S = pwwkewxpw

left, right

Set = [k, e, w, x]

right ++

S = p w w k e w x p w
left ↑ right ↑

Set = [k, e, w, x, p]

right++

max l = 3

S = p w w k e w x p w
left ↑ right ↑

w is already present in set
so,
length = right − left
length with no duplicate = 8 − 3
= 5

max l = 5

remove k from Set = [e, w, x, p]

left++

S = p w w k e w x p w
left ↑ right ↑

duplicates in set due to w character
so remove w from set

S = p w w k e w x p w
left ↑ right ↑

remove e ⇒ [w, x, p] Set

left++

S = p w w k e w x p w
left ↑ right ↑

remove w ⇒ [x, p] Set

left++

S = p w w k e w x p w
left ↑ right ↑

Set = [x, p]

S = p w w k e w x p w
left ↑ right ↑

Set = [x, p]

at right we have w
here w is not present in
set so insert w in set

Set = [x, p, w] → length = 3

right++

Hence, max len = 5

T.C = O(n) → one iteration
S.C = O(1)

S = p w w k e w x p w
left ↑ right ↑

right is out of bound so, stop here.

Code

```
~~class~~ Solution {
public:
    int lengthofLongestSubstring(string s){
        unordered_set<char> charSet;
        int maxLength=0;   int left=0;

        for(int right=0; right<s.length(); right++){
            while(charSet.count(s[right])) {
                charSet.erase(s[left]);
                left++;
            }
            charSet.insert(s[right]);
            maxLength = max(maxLength, right-left+1);
        }
    }  return maxLength;
};
```